AutoSQUID Operating Protocol

Version 0.2.0

Ran Laboratory, Washington University in St. Louis

2026-06-11

Abstract

`AutoSQUID` is a Python package for automated noise-trace acquisition with STAR Cryoelectronics PFL-102 SQUID electronics and an NI data-acquisition card. It collects clean, gap-free time series at selected sample intervals, detects flux-lock failures, resets and verifies the flux-locked loop, optionally records mixing-chamber temperature, and saves traces in the standard `PCS102` text format with append-only run and action logs. This document is the operating protocol for that system: it defines the required operator actions, the data-quality acceptance criteria a trace must meet to count as science data, and the records that make each dataset reproducible. The reusable acquisition, control, analysis, and tuning logic lives in the package; small example notebooks provide the operating workflow, driven by a single `Config` object that holds hardware channels, acquisition settings, output paths, and an optional site-supplied thermometer reader. Package internals — the locator algorithm, run counters, and a worked log example — are collected in the appendices for maintainers.

Document control

Document	AutoSQUID Operating Protocol (installation, operation, and data quality)
Software version	0.2.0 (Git tag `v0.2.0`)
Release date	2026-06-11
Maintainer	Zhengyue Chen (`czhengyue@wustl.edu`)
Principal investigator	Sheng Ran (`rans@wustl.edu`)
Institution	Department of Physics, Washington University in St. Louis
Repository	`https://github.com/zhengyuechen/AutoSQUID`
Applies to	STAR Cryoelectronics PFL-102 (gain 5040) + PCI-1000 + DAC-2568, read by an NI PCIe-6320 (`Dev1`, 16-bit, 250 kS/s) on `ai0` in **DIFF** mode at ±1 V; SCC control on `COM3`; SQUID-locked, high-sensitivity register `0x408` (100 kΩ / 1.5 nF). Validated 2026-06 on the Ran Lab Bluefors rig.

Contents

1 Purpose, scope, and audience

1.1 Purpose

This protocol specifies how to install, operate, and verify the data quality of the `AutoSQUID` system for measuring SQUID output-voltage noise time series. It defines the operator actions required, the acceptance criteria a trace must meet to count as science data, and the records that make each dataset reproducible and auditable.

1.2 Scope

In scope: configuration of the `AutoSQUID` package; NI PCIe-6320 analog input and sampling; PFL-102 serial control and software reset; the measurement state machine; the operating procedure and its go/no-go checks; data-quality classification and acceptance; and the per-dataset reproducibility record.

Out of scope: cryostat and dilution-refrigerator operation; sample mounting and thermometry calibration; physical (re)wiring of the rig; the manual SQUID tune-up and per-cooldown flux calibration performed in PCS102DA; and downstream PSD/analysis beyond the integrity gate. These are prerequisites or follow-on steps performed outside this protocol.

1.3 Audience and required familiarity

The operator is a trained laboratory member who can lock the SQUID and set its sensitivity in PCS102DA, read the mixing-chamber thermometer, and run a Python/Jupyter notebook. Assumed familiarity: basic SQUID flux-locked-loop operation (lock vs. tune, reset, rail), NI-DAQmx terminal configuration (RSE/NRSE/DIFF), and the per-cooldown flux-calibration concept. No knowledge of the package internals is required to operate it; the internals are documented in the appendices for maintainers.

1.4 Validated hardware and configuration

This release was validated on the configuration in the *Document control* block above: STAR Cryoelectronics PFL-102 (gain 5040) + PCI-1000 + DAC-2568, read by an NI PCIe-6320 on `ai0` in **DIFF** mode at $\pm 1\,\mathrm{V}$, with SCC control on `COM3` and the SQUID-locked, high-sensitivity register `0x408` ($100\,\mathrm{k\Omega}$ / $1.5\,\mathrm{nF}$). Other lock states, channels, ranges, or point goals (20 M, ...) are supported through `Config` but were not part of this validation; re-verify the go/no-go checklist (Section 6) after any such change.

2 Configuration parameters

All run settings are fields of one `Config` object, built in the notebook's first cell (`cfg = sq.Config(...)`); every library function takes that `cfg`. The table below explains each field; the defaults shown are the shipped values. A few derived values are computed properties of `cfg` (the

scan-interval list, the file-name tag, the output directory, and the PFL register), so there is no duplicated state to keep in sync.

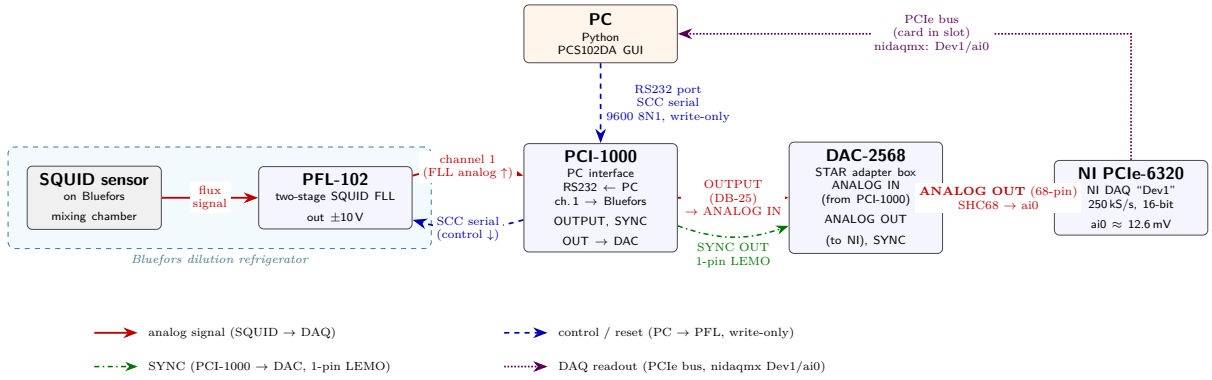


Figure 1: Wiring of the SQUID acquisition rig. Analog: SQUID/PFL-102 (Bluefors) → PCI-1000 ch.1 → PCI-1000 OUTPUT → DAC-2568 ANALOG IN → ANALOG OUT (68-pin) → PCIe-6320 ai0, read over the PCIe bus via nidaqmx. Control: PC → PCI-1000 RS232 → PFL (SCC serial, write-only). Sync: PCI-1000 SYNC OUT → DAC-2568 (1-pin LEMO).

`cfg` field	Meaning	Default
Acquisition		
`scan_interval_s`	Sample interval(s) in seconds; one full cycle runs per entry (a single number or a list). Sets the sample rate $f_s = 1/\text{interval}$ (4/20/100/500 μs → 250/50/10/2 kHz).	`[100e-6]`
`n_points`	Number of samples in each consecutive run.	`10_000_000`
`temp_label`	`"auto"` reads, validates, and rounds the current MXC temperature into the file name (a missing / non-finite / non-physical reading aborts before acquiring); or a literal such as `"14mK"`.	`"auto"`
`n_trials`	Target number of *clean* traces to collect per scan interval.	`2`
Live jump check (during acquisition)		
`chunk`	Points per streamed read; this is the jump-check cadence. It does *not* break the gap-free run (see Section 3).	`100_000`
`jump_v`	Baseline-mean slip threshold (V): below the ±1 V clip, above clean drift.	`0.5`
`rail_v`	True-rail catch, $\lvert V\rvert >$ this. Kept at 9.5 (range-agnostic); at ±1 V the ADC clips below it, so a clipped slip reads as a jump.	`9.5`
`baseline_chunks`	Opening chunks that set the baseline mean μ_0 (the first chunk).	`1`
`live_jump_check`	Run the mid-run jump/rail abort during acquisition. `False` drains each run without it; the clean prefix is then located post-hoc (Section 5).	`True`

cfg field	Meaning	Default
min_usable_frac	A truncated clean prefix counts as a clean (*accepted*) trial only if its usable points ≥ this fraction of n_points; 1.0 counts only full-length clean traces.	0.90
Temperature logging		
temp_logger	Run the background TempLogger thread during acquisition. False skips it entirely: no thread, no TEMP_*.csv, and no temp_reader needed for a literal temp_label.	True
temp_every_s	Background temperature sampling period, in seconds.	30
temp_channel	Thermometer channel passed to temp_reader; 6 = mixing chamber (MXC).	6
temp_reader	*Installation-specific*, injected in the notebook: fn(channel) -> T in K (wrap the local thermometer API). None until set; required for any run with temp_label="auto".	None
Serial control (SCC)		
port	Serial COM port for control. PCS102DA can stay open on COM2 / another channel to hold S-lock.	"COM3"
channel	SCC address = the locked PFL channel.	1
reset_opcode	SCC opcode for the feedback-loop register.	0x50
baud	Serial speed (8N1: 8 data bits, no parity, 1 stop bit).	9600
register_override	Force a specific PFL register; None uses the standard SQUID-locked 0x408.	None
Verify / DAQ / output		
verify_n, verify_fs	Samples and rate of the post-reset verify read.	1000, 10_000
scan_n, scan_fs	Samples and rate of the channel-liveliness probe.	4000, 50_000
baseline_v	|mean| below this counts as a cleared / locked baseline.	0.10
vrange	DAQ analog-input range, in volts (Section 3).	1.0
terminal_config	NI analog-input reference (Section 3): "DIFF" (vs the paired negative input ai0/ai8, rejects common-mode; "DIFFERENTIAL" accepted as an alias), "RSE" (vs board ground), or "NRSE" (vs AI SENSE).	"DIFF"
max_attempts	Maximum total acquisitions per interval (clean + failed).	4
force_device, force_ai	Override the device / channel auto-pick.	None
daq_ai	NI analog-input carrying the output; empty until the channel auto-detect (detect_ai_channel) sets it, or set force_ai.	""
data_root	*Installation-specific* data root; outdir = data_root / user / date. Set per install.	""
user	Operator name; selects the data folder.	""
date	Date subfolder ("" = the bare user folder).	""

Estimated time per interval ≈ n_points × interval (≈1000 s at 100 μs, ≈5000 s at 500 μs). One notebook run covers one temperature setpoint and all the scan intervals listed; a new temperature is a new run. Traces are written in the STAR Cryoelectronics PCS102 text format [5].

3 The PCIe-6320 DAQ card

A National Instruments PCIe-6320 (16-bit multifunction DAQ, up to 250 kS/s) [1] digitizes the analog flux-locked-loop output. The signal reaches the card on `ai0` via the PCI-1000 output and the DAC-2568 adapter. Because the maximum rate is 250 kS/s, the 4 μs scan interval (250 kHz) sits exactly at the card's ceiling. Acquisition uses the NI-DAQmx API [2].

3.1 Input range (`vrange`) and ADC resolution

`vrange` is the *card's* programmable analog-input range, not a PFL-102 setting. It is set to ± 1 V — the range the NI card was set to for the earlier measurements on this rig. The 6320 offers four discrete ranges — ± 0.2, ± 1, ± 5, and ± 10 V — and the driver rounds the requested interval up to the smallest range that contains it.

The converter is 16-bit, so it always produces $2^{16} = 65{,}536$ codes *regardless* of range; the range is simply the voltage span those fixed codes are stretched across:

$$\text{LSB} \;=\; \frac{2 \times \texttt{vrange}}{65{,}536}.$$

A smaller range therefore gives finer steps (± 10 V $\rightarrow \sim 305\,\mu$V/code; ± 1 V $\rightarrow \sim 30.5\,\mu$V/code) at the cost of headroom. It is a strict dynamic-range vs. resolution trade at fixed bit depth.

Caution. This notebook uses ± 1 V (the range the NI card was set to for the earlier measurements). The clean signal is ~ 12 mV mean with ~ 2 mV noise, so ± 1 V gives $\sim 30\,\mu$V/code — $\sim 10\times$ finer than ± 10 V — with ample headroom over the signal. The trade is that the ADC *clips* at ± 1 V, so the thresholds account for it: `jump_v` is lowered to 0.5 V to catch an in-range flux-slip by its mean deviation, while `rail_v` stays at 9.5 V — at ± 1 V the ADC clips at ~ 1 V (below 9.5), so the rail check never fires and a clipped slip is reported as a jump; the same notebook then still flags a true rail if the card returns to ± 10 V.

3.2 Terminal configuration: DIFF (`cfg.terminal_config`)

The reference the ADC measures `ai0` against is set by `cfg.terminal_config` (`"RSE"` / `"NRSE"` / `"DIFF"`; default `"DIFF"`). **DIFF (differential)** reads `ai0` against its paired negative input (`ai0`/`ai8`), so common-mode / ground-loop pickup cancels — the right match for the battery-powered (floating) SQUID readout, and the mode the PCS102DA software uses. *RSE* (Referenced Single-Ended, vs the board's AIGND) and *NRSE* (vs the shared AI SENSE line) are the alternatives.

3.3 Sampling: one gap-free run, drained in chunks

Each acquisition is a *finite* task with `samps_per_chan = N_POINTS`: one hardware-timed sample clock fills a single buffer for the whole run. That buffer is then drained in `chunk`-point reads so the live jump check can run every `chunk` points. Chunking is only a read pattern — adjacent chunks are one sample period apart — so the run remains a single, consecutive, gap-free acquisition. Because the buffer holds the entire run, a slow read cannot cause an overflow. When a live check fails the

task is stopped early (before all `N_POINTS` are read); the NI warning `200010` that this raises is intentionally suppressed in that one place, since the early stop is deliberate.

4 The PFL-102 and SCC serial control

The PFL-102 is the two-stage SQUID flux-locked loop (FLL); its buffered output has a range of $\pm 10\,\mathrm{V}$ (voltage gain 5040) and reaches the DAQ through the PCI-1000 and DAC-2568. Control is one-directional: the PC sends commands down the PCI-1000 RS-232 link; there is no readback. The serial command set is the STAR Cryoelectronics SCC protocol [3], reimplemented from the OpenSQUID reference [4].

4.1 Lock and rail states

The PFL has two relevant states per stage, **LOCK** and **TUNE**:

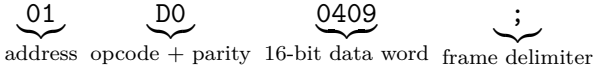
- In **TUNE** the feedback loop is *open* (used to set up the SQUID). The output then shows the SQUID's bounded, periodic voltage–flux characteristic — a small modulated signal, *not* a saturated rail. The noise can even be measured directly in this state.
- In **LOCK** the output tracks the applied flux. A **RESET** only has an effect in LOCK mode: it briefly opens the loop and discharges the integrator so tracking restarts as close to zero as possible.

Consequently, an unlocked output is small and bounded; a *railed* ($\sim \pm 10\,\mathrm{V}$) output is a *locked* loop that has run out of feedback range — the integrator driven past the $\pm 10\,\mathrm{V}$ buffer limit by large applied flux or drift, an "off-scale" condition cleared by a RESET. The automation's reset-and-recover logic therefore assumes the channel is locked; if a channel dropped to TUNE, a RESET would do nothing.

4.2 The SCC serial protocol

Commands are sent as ASCII text over a write-only link at 9600 baud, 8 data bits, no parity, 1 stop bit (8N1).

Command frame. Two hex address digits, two hex opcode digits, four hex data digits, ending in a semicolon — format `"%02X%02X%04X;"`:



Parity. Odd parity over the set bits of (address + opcode + data), placed in the most-significant bit (bit 7) of the opcode byte. The low seven bits of that byte are the opcode itself.

Addressing. A PFL on channel N has SCC address N (channel 1 = `0x01`); the PCI-1000 itself is `0xFF`.

4.3 The reset command

A reset is two frames: *assert* (data = register with bit 0 set) then *release* (data = register, returning to locked operation). With channel 1, opcode `0x50`, and the standard register `0x408`, the bytes on the wire are:

	frame on the wire	meaning of bit 0
assert	`01D00409;`	reset = on
release	`01500408;`	reset = off (locked operation)

(The parity bit makes the opcode byte `0xD0` for the assert frame and `0x50` for the release.) The reset toggles only bit 0 and preserves the rest of the register — it does not change any DAC values.

The feedback register (`0x50` data word). The standard configuration is `0x408`. Decoded against the PFL-102 register bit-fields (below), the only set bits are bit 10 and bit 3, giving: reset off; feedback resistor 100 kΩ; integrator 1.5 nF; test input off; and input-SQUID *Lock* with array *Tune* — i.e. SQUID-locked, high sensitivity, test off.

Table 2: PFL-102 feedback-loop register (opcode `0x50`).

Bits	Field	Encoding
`0`	Reset	`0` = off, `1` = reset
`2,1`	Feedback resistor	`00` = 100 kΩ, `01` = 1 kΩ, `10` = 10 kΩ
`9,8`	Integrator	`00` = 1.5 nF, `01` = 15 nF, `10` = 150 nF
`7-4`	Test input	`0000` = off, `0001` = SQUID bias, `0010` = array bias, `0100` = SQUID flux, `1000` = array flux
`3` and `11,10`	Tune/Lock	SQUID Lock + array Tune: `11,10`=`01`, bit 3=`1`. SQUID Tune + array Lock: `01`, `0`. Both Tune: `10`, `0`.

Caution. Confirm `cfg.register` (set via `register_override`, else the standard `0x408`) and `channel` match the actual front-panel lock state before each run. The release frame writes the *whole* register back, so a mismatch silently reprograms the PFL instead of merely resetting it. For array-locked operation, a different sensitivity, or a different channel, set `register_override` / `channel` accordingly.

4.4 Detecting a failed reset

Two independent checks guard the reset:

1. **Verify read.** Immediately after the reset, a short read confirms the output cleared (|mean| < `baseline_v`). If it never clears after several retries, the run is a reset failure and the sweep stops.
2. **Baseline-at-start.** The opening chunks of each acquisition are watched; if the output is off-zero there, the reset "did not hold" and the run is flagged as a bad baseline.

The first asks *"did the reset clear it now?"*; the second asks *"is it still clear when the run begins?"* A slipped lock that briefly looked clear during the verify read is caught by the second check.

Resets fire after every *slipped* acquisition (a truncated or non-clean trace), and one further **exit reset** runs if the *final* accepted trace itself left the lock slipped — so an interval always returns with the lock cleared, ready for the next interval. Every reset (in-loop or exit) goes through the same verify; if any of them never clears within its retries, `run_cycle` returns `"reset_fail"` and the whole sweep stops. A full clean trace leaves the lock intact and fires no reset.

5 Structure of the run

Within a single scan interval, `run_cycle` repeats the loop below until it has collected `n_trials` *accepted* traces or used its acquisition budget (`max_attempts`). The setup before the loop — resuming from any traces already on disk, then the initial reset — is omitted here. An interval that already has `n_trials` accepted traces on disk is skipped immediately, before any reset or other hardware action.

```
repeat until  (N_TRIALS accepted)  OR  (MAX_ATTEMPTS acquisitions used):

    acquire one consecutive run     # N_POINTS @ fs
         |     # live flag fires -> cut at tripping chunk (buf[:chunk_start])
         v     # no live flag    -> locate clean prefix post-hoc (gap_tol)
    evaluate for the clean pre-failure prefix
         |
    classify -> event    (JUMP|RAIL|FROZEN|DEAD|BAD_BASELINE|NONE)
                outcome  (CLEAN|SURGE|DEAD|BAD_BASELINE)
                accepted = CLEAN and usable >= MIN_USABLE_FRAC*N_POINTS
         v
     < outcome? >
         |
         +-- CLEAN ........ save DAQ_..._{usable}pts_{i}.txt (+TEMP); log
         |                  n_attempts++ ; if accepted: n_clean++
         |                  full-length -> loop again  (lock good, no reset)
         |                  truncated   -> reset_and_verify, then loop
         |
         +-- SURGE ........ save ..._{i}_SURGE.txt ; log ; n_attempts++
         |                  -> reset_and_verify (cleared? yes->loop; no->STOP)
         |
         +-- DEAD ......... NO file (0 usable pts) ; log ; n_attempts++
         |                  -> reset_and_verify (cleared? yes->loop; no->STOP)
         |
         +-- BAD_BASELINE . save ..._{i}_BADBASE.txt  (reset did not hold)
                            -> STOP the whole sweep

after the loop:  final accepted left lock slipped -> EXIT reset_and_verify
                 n_clean >= N_TRIALS ?  yes -> DONE
                                        no  -> BUDGET_EXHAUSTED (next interval)
```

Here `i` is the on-disk order index: it increments on *every* acquisition, so a re-run resumes without overwriting. After the initial reset, additional resets fire only after a *slipped* acquisition — a truncated or non-clean trace — never after a full clean one; and if the *final* accepted trace itself left the lock slipped (its prefix was truncated), an **exit reset** runs through the same `reset_and_verify` before the interval returns, so the lock is always left clean. A reset that will not clear — or a bad

baseline at the start (the reset did not hold) — is systemic and stops the whole sweep (`run_cycle` returns `reset_fail`).

Every acquisition is evaluated for a usable clean prefix; a full clean trace remains unchanged. When a live jump/rail flag fires the boundary is taken at the tripping chunk; when a run finishes without a live flag the prefix is located post-hoc. The locator algorithm, its `gap_tol` persistence, the run counters, and the detailed event semantics are in Appendix A; the operator-facing acceptance rule is the classification table in Section 8.

With `temp_logger=False`, no background thread or per-trace `TEMP_*.csv` is created, and the ledger's start/end temperatures are `nan`.

6 Pre-run go/no-go checklist

Complete every check below *before* starting an unattended acquisition. Any **No-go** condition must be resolved before proceeding; the detailed procedure follows in Section 7.

Check	Go criterion	No-go → action
Wiring & channel	`detect_ai_channel` identifies exactly one live channel carrying the SQUID output on `ai0`	No live channel → fix lock/wiring; set `force_ai` if known
Lock & register	SQUID-locked, high sensitivity; `cfg.register` / `channel` match the front panel	Mismatch → correct `register_override` / `channel`
Temperature read	`sq.read_temp(cfg)` returns a physical value (or a literal `temp_label` is set with `temp_logger=False`)	Missing / non-physical → fix reader or channel
Baseline & reset	`reset_and_verify` reports `CLEARED` ($\lvert\text{mean}\rvert < 0.10$ V)	`RESET_FAIL` → stop; check port / register / lock
Auto-S-tune	`status = converged` (or `dac_limited` acceptable for the run)	`no_response` → re-tune; check lock / `daq_ai` (Section 10)
DAQ sample count	A short *and* a full finite task each return the exact requested `n_points`	Short count / DAQ error → run the DAQ smoke test
Output directory	`cfg.outdir` exists and is writable	Missing → create / fix `data_root` / `user` / `date`

Halt conditions during a run: any `RESET_FAIL`, `BAD_BASELINE`, loss of the live channel, or `no_response` stops the sweep — investigate before resuming (the run resumes from disk, never overwriting).

7 Operating procedure

Caution. The finite-acquisition path is new on this hardware. Run the one-time bring-up (Section 7.1) as a supervised dry run before trusting it for science data; validate each workflow step before unattended acquisition.

7.1 One-time bring-up

Do these once, or whenever the rig or PC changes. Each is a configuration issue that can invalidate an acquisition.

- **Package import.** `import AutoSQUID as sq` (install once with `pip install AutoSQUID`); the acquisition PC has the required DAQ/serial dependencies (`nidaqmx`, `pyserial`) and the local thermometer backend installed.
- **Serial port.** `cfg.port` defaults to `COM3`; confirm it with the hardware-validation notebook. PCS102DA can stay open on a different port (e.g. COM2) to hold the SQUID lock — just not on `COM3`.
- **Temperature backend.** Confirm `sq.read_temp(cfg)` returns the MXC value in kelvin and that channel 6 is the mixing chamber; compare to the Bluefors display.
- **DAQ smoke test.** Confirm a finite `samps_per_chan = 10_000_000` task is accepted and returns real (non-zero) data. Run one short acquisition first (e.g. `n_points = 100_000`), then one full 10 M run; confirm the full sample count comes back.
- **Save root.** Confirm the data root folder for the configured `user` exists (`cfg.outdir`).

7.2 Before every run (per temperature)

1. **Lock the FLL** on the selected channel: SQUID-locked (stage 1), high sensitivity (100 kΩ / 1.5 nF). This is the per-cooldown manual calibration. Acquisition reliability requires a stable lock.
2. **Confirm `cfg.register` / `channel` match that lock** (Section 4.3).
3. **Baseline near zero.** With the loop locked, the output should sit within ±0.1 V of zero so the post-reset verify passes and the trace is centered. Maintain the baseline near zero.
4. **Set and settle the temperature.** The label and the start temperature are read at the moment the run begins.
5. **Choose the temperature mode.** With `temp_logger=True`, confirm the reader returns a fresh value. With `temp_logger=False`, use a literal `temp_label`; no background temperature thread or per-trace CSV will be created.
6. **Keep PCS102DA off this script's COM port.** It can stay open to hold the SQUID lock as long as it is on a different port (e.g. COM2) than this script (`COM3`); two programs on the same port collide.

7.3 Run order

Run the workflow in the order below; do not "Run All" on a new setup. Each step has an expected result and a stop condition — if the result is not met, halt and resolve it before continuing.

#	Operator action	Expected result	Stop condition
1	Confirm wiring (Fig. 1); lock the input SQUID (`sq.s_lock(cfg)` or PCS102DA)	`ai0` carries the output; channel locked	No live channel
2	Center the output: `sq.auto_s_tune(cfg, start_sflux=..., target_V=0.0)`	`status = converged` (or acceptable `dac_limited`)	`no_response`
3	Prepare the config: `data_root`, `user`, `date`, `scan_interval_s`, `n_points`, `n_trials`, `port`, `terminal_config`, temperature mode	Required fields set	A required field unset (raises)
4	Detect the input: `dev, cfg.daq_ai = sq.detect_ai_channel(cfg)`	Exactly one live channel	None found
5	Resolve the label: `sq.resolve_temp_label(cfg)`	Rounded `temp_label` set	Bad / no MXC read
6	Acquire interval by interval (loop below)	`n_trials` accepted traces per interval	`run_cycle` returns `reset_fail`
7	Watch the console and logs	`event`/`outcome`/`accepted` lines; logs written	`RESET_FAIL` / `BAD_BASELINE`

Steps 2, 4, and 6 use these commands (the acquisition loop stops the sweep on the first `reset_fail`):

```
res = sq.auto_s_tune(cfg, start_sflux=..., target_V=0.0)   # step 2
dev, cfg.daq_ai = sq.detect_ai_channel(cfg)                # step 4
for tau in cfg.scan_intervals:                             # step 6
    if sq.run_cycle(cfg, tau) == "reset_fail":
        break
```

`run_cycle()` creates the output directory, verifies/reset-checks the lock, acquires traces, writes the PCS102 files, logs temperature when enabled, and resumes from existing files by date-agnostic trace core. The temperature label is resolved *once* (step 5); the temperature log is then sampled afresh for every acquisition, and an individual failed read is recorded as `nan` in that trace's CSV rather than stopping the run. For a resumed run, set `temp_label` to the same literal label the original run produced.

7.4 During the run

Each interval acquires until it has `n_trials` *accepted* traces or exhausts `max_attempts` acquisitions. Every acquisition is evaluated for a usable clean prefix and classified on three axes recorded in the ledger: `event` (what triggered a cut), `outcome` (integrity of the saved prefix), and `accepted` (whether it counts toward `n_trials`). The formal mapping — valid science trace, saved diagnostic, rejected acquisition, or systemic failure — is the acceptance table in Section 8; the full event-by-event meaning, the run counters, and a worked console/ledger example are in Appendix A and Appendix B.

The console prints one line per acquisition with these fields plus the start and expected-done times. After a reset, `reset_and_verify` prints `CLEARED` on the first of up to five tries that brings |mean| below `baseline_v` (< 0.1 V); a failure prints `RESET_FAIL`. Long silent stretches during acquisition are normal — the background temperature samples are written to the CSV rather than printed, and a 5000 s run is not frozen.

Stopping mid-run. Interrupting the kernel during an acquisition stops at the next chunk boundary — after the current `chunk`-point read returns, not instantly. The in-progress trace is *discarded, not written* (only acquisitions that finished this run are on disk), and no reset is fired.

7.5 After the run

1. **Check the output folder** for the expected accepted traces — named with a month-day date prefix and the trace's *usable* point-count tag (e.g. `DAQ_Jun01_4us_4K_10Mpts_1.txt` for a full trace, or an `8p4Mpts`-style tag when truncated). Each has its temperature CSV when `temp_logger` is on, plus any `_SURGE` / `_BADBASE` traces, the run ledger (`experiment_log.txt`), and the results-free event log (`action_log.txt`). The output should contain `n_trials` accepted (bare-suffix) files per interval.

2. **Read the ledger** (`experiment_log.txt`): one tab-separated row per acquisition. The columns are `timestamp`, `scan_interval_us`, `n_target`, `n_clean` (running accepted count), `attempt` (the on-disk order index), `event`, `outcome`, `usable_points`, `usable_seconds`, `accepted`, `n_resets`, `mean_V`, `std_V`, `T_start_K`, `T_end_K`, and `filename` (empty for a `DEAD` row). `event` (the trigger) and `outcome` (the integrity of the saved prefix) are separate axes, and `accepted` marks the rows that count toward `n_trials`. `n_resets` counts reset/recovery *calls* this interval, not their internal retries: each call runs up to five reset+verify attempts, and the `action_log.txt` `RESET` line records how many actually cleared the baseline (e.g. `RESET,2 tries`). Each row's `n_resets` is the count *as of that ledger row*: the **exit** reset that fires when the final accepted trace left the lock slipped happens *after* the last row is written, so it appears only in `action_log.txt`, never in a ledger `n_resets`. The companion `action_log.txt` records only the event stream — resets, the temperature read just before each measurement, each `MEASURE` attempt, the `S_FLUX` tune result, and bad-baseline — never any trace data.

3. **Re-check integrity before downstream analysis.** Acquisition already screens each trace with `is_surge_spec` before labeling it CLEAN — rails, $> 6\sigma$ baseline jumps, and frozen/stuck flat segments (flagged when a chunk's std collapses below the live noise level) — but a standalone re-check before PSD/analysis is recommended.

4. **Complete the reproducibility record** (Section 9): traces are saved in raw volts with no flux-calibration factor or sample ID, so record the cooldown, sample, and Φ_0/V calibration factor (with its calibration date) alongside the dataset.

5. **To collect more traces or finish an interrupted interval**, re-run the active cell with the same settings. Resume counting comes *from the ledger*: the number of `accepted=1` rows for this core whose trace file still exists on disk (so it uses the exact `usable_points` written at the time, immune to filename rounding). The matching core is the interval and temperature only — the month-day prefix *and* the usable point-count tag are ignored — so traces from *any*

day and any usable length all count. The next index is the highest index on the *filesystem* +1, so a resume never overwrites any existing trace. The temperature label is part of the core, so when automatic labeling was used set `temp_label` to the literal label the run produced before resuming.

7.6 Troubleshooting

Symptom	Likely cause → fix
Reset is not working / `RESET_FAIL` in the ledger; sweep stopped	A reset would not hold (this is by design systemic). Check the port (`COM3`; make sure PCS102DA is not on it) and that `cfg.register`/`channel` match the lock; confirm the lock and cable. Fix, then re-run (resumes from disk).
DAQ error, or the trace is all zeros	Finite 10 M buffer rejected, incorrect range, or a copy-vs-view read issue — run the DAQ smoke test.
No live channel in the checks cell	SQUID not locked, incorrect device, or output not on this card — confirm the lock and wiring; set `force_ai` when the channel is known.
Run crashes at the temperature-label line	The temperature reader returned nothing (reader unset / wrong channel). Set it, or use a literal label with `temp_logger=False`.
Fewer clean files than `n_trials`	The interval hit the attempt budget (lock kept slipping). Re-run to top up, or raise `max_attempts`.
A bad-baseline trace and the sweep stopped	The reset is not holding (same causes as a reset failure). Fix the reset, then re-run.
No console activity during acquisition	Long silent stretches are normal; temperature samples go to CSV rather than the console, and a 500 μs run is ∼83 min.

7.7 Known limitations

- At the ±1 V range the ADC clips at ∼1 V, so any excursion past ∼1 V reads as a ∼1 V step and is recorded as an `event=JUMP` (the clean prefix before it is kept; the post-jump tail is discarded). `rail_v` stays at 9.5 (unreachable here, so a `RAIL` event is dormant at ±1 V) and remains valid if the card returns to ±10 V.
- Non-clean traces (`_SURGE` / `_BADBASE`), any enabled temperature CSVs, the ledger, and the action log live in the same folder as accepted traces; when batch-loading, exclude the suffixed traces and the logs. Accepted traces are exactly those with a bare integer index before `.txt` (no outcome word); matching ignores the month-day date prefix *and* the point-count tag (keyed on the interval/temperature core). Note that a bare-suffix file can still be a truncated prefix below `min_usable_frac` (logged `accepted=0`); the ledger's `accepted` flag, not the filename, is authoritative for which traces counted.
- One temperature label is baked into all intervals of a run; on a long multi-interval sweep the label can drift from the true value — the per-trace start/end temperatures in the ledger are authoritative.

8 Data-quality classification and acceptance

Every acquisition is classified on three axes recorded in the ledger: `event` (what triggered a cut), `outcome` (integrity of the saved prefix), and `accepted` (whether it counts toward `n_trials`). The table below is the formal acceptance criterion; it distinguishes a valid scientific trace, a saved diagnostic, a rejected acquisition, and a systemic failure that halts the sweep. `accepted=1` requires `outcome=CLEAN` *and* usable points $\geq$ `min_usable_frac` $\times$ `n_points`.

Category	event	outcome	file	Disposition
Valid science trace	NONE / JUMP / RAIL / FROZEN	CLEAN	bare `..._-npts_-i.txt`	`accepted=1`; counts toward `n_trials`
Diagnostic: short clean prefix	JUMP / RAIL / FROZEN	CLEAN	bare (tag < target)	`accepted=0`; kept, not counted; reset & retry
Diagnostic: surged	any	SURGE	`_SURGE`	`accepted=0`; kept for inspection; reset & retry
Rejected: dead	DEAD	DEAD	none	`accepted=0`; not saved; reset & retry
Systemic: bad baseline	BAD_-BASELINE	BAD_-BASELINE	`_BADBASE`	STOP the sweep; reset is not holding
Systemic: reset failure	—	RESET_-FAIL	none	STOP the sweep; `run_cycle` returns `reset_fail`

Only `accepted=1` bare-suffix files are science data. A bare-suffix filename alone is *not* sufficient: a truncated prefix below `min_usable_frac` is also bare-suffix but logged `accepted=0`. The ledger's `accepted` flag is authoritative for which traces counted (Section 9).

9 Scientific traceability and reproducibility record

Each dataset must be reproducible from its records alone. The table lists what to record and where it is captured. Items marked *auto* are written by the package; items marked *manual* must be recorded by the operator — the package stores raw volts only and knows nothing of the sample, cooldown, or calibration.

Record	Source	Where / note
AutoSQUID version + Git commit/tag	manual	e.g. `0.2.0` / `v0.2.0`; pin in the dataset README
Complete `Config`	auto/manual	all `cfg` fields; save the notebook config cell
Operator, sample ID, cooldown ID	manual	dataset README
Temperature	auto	`temp_label` + per-trace `T_start_K` / `T_end_K` in the ledger (authoritative)

Record	Source	Where / note
Hardware / channel configuration	manual	device, `daq_ai`, `terminal_config`, `vrange`, `register`, `port`
Flux-conversion calibration	manual	Φ_0/V factor *and* its calibration date / cooldown (not stored with the traces)
Per-acquisition record	auto	`experiment_log.txt` (Appendix A) + `action_log.txt`
Deviations from this protocol	manual	note any departure (non-default thresholds, `live_jump_check=False`, ...)

10 Auto-S-tune: centering the locked output

Before acquiring, the locked output is centered near zero so the baseline-near-zero precondition (Section 7) is met. Manually this is done by nudging the SQUID-flux control until the continuously-acquired trace looks centered around 0 mV. `auto_s_tune` (in `example_auto_s_tune.ipynb`) automates exactly that.

10.1 What it does

With the input SQUID *already locked*, it steps the SQUID-flux DAC (the `set_squid_flux` current, SCC `0x60` sub-opcode `0x0`) and, after each step, takes a short *finite* "live-view" read (`live_mean`, default 1 s) of the output mean. It moves toward the target by a **secant** update,

$$\Delta x \;=\; \frac{V_{\text{target}} - \bar{V}}{\text{slope}}, \qquad \text{slope} = \frac{\bar{V}_n - \bar{V}_{n-1}}{x_n - x_{n-1}} \text{ (V per } \mu\text{A)},$$

so the sign and gain of the response — unknown and per-cooldown — are measured, not assumed. Each step is clamped to `max_step_uA` so the lock cannot slip a fluxon. The first response probe is 10% below `start_sflux`; if that side is flat, the tuner retries once 10% above the same start point before declaring `no_response`. This mirrors OpenSQUID's `zeroStage1Output` [4], and uses the same finite DC reads (not a continuous stream). There is *no reset between steps*: while locked, the flux shifts the baseline directly — matching both the manual workflow and OpenSQUID.

10.2 Stop conditions

It stops when the live mean is within `tol_V` of target (default 20 mV — the "looks centered" criterion; on a 1 s read the mean's standard error is ∼0.02 mV, so this band is stable, not chasing noise), confirmed by one further read to reject slow drift. As a fallback it also stops when the predicted step falls below one DAC LSB (∼0.024 μA) — it cannot move finer (status `dac_limited`). It returns a status dictionary: `status` ∈ {`converged`, `dac_limited`, `no_response`, `max_iter`}, plus `flux_sflux`, `mean_V`, `std_V`, and `n_iter`.

10.3 Procedure

1. Close PCS102DA on the control port (or keep it on a different port).
2. In `example_auto_s_tune.ipynb` §0, build `cfg` (`port`, `channel`, `daq_ai`, `vrange`, `data_root`, and `user`). The tune writes its result to the run's `action_log.txt`.
3. §1: `sq.s_lock(cfg)` to lock the input SQUID (or lock in PCS102DA), and make sure the trace is already *roughly* centered — within the ±1 V range. The default `reset_first=True` performs one pre-reset; set it to `False` to tune from the live locked state.

4. Run `res = sq.auto_s_tune(cfg, start_sflux=..., target_V=0.0)`. Check `res["status"]`: `converged` means the output now sits within `tol_V` of zero at the returned `flux_sflux` (`dac_limited` means as close as one DAC LSB allows, still outside `tol_V`). `no_response` after the down/up probes means to check the lock, `daq_ai`, and S-flux path.

A Detector internals: locator, counters, and event semantics

This appendix documents the package internals behind the run loop (Section 5) and the acceptance table (Section 8). It is reference material for maintainers; operators need only the acceptance table.

A.1 Truncation and the post-hoc locator

Every acquisition is *evaluated* for a usable clean prefix; a full clean trace remains unchanged. When a live jump/rail flag fires, its exact boundary is final and the locator is *not* called: the chunk that tripped is dropped and the strict prefix before it (`buf[:chunk_start]`) is kept — no helper re-interprets the cut. When an acquisition completes *without* a live flag — whether `live_jump_check` was on (a clean finish) or off (the run drained with no mid-run abort) — the prefix is located *post-hoc* by the persistent locator `usable_points_from_spec`: it scans every chunk with *absolute* thresholds (a rail, $|\mu - \mu_0| >$ `jump_v`, or a frozen chunk versus a robust 95th-percentile noise level) and cuts at the *onset of the persistent failing run that reaches the end*, walking back from the end and permitting up to `gap_tol` *consecutive* clean chunks (internal default `3`); recovery requires *more than* `gap_tol` consecutive clean chunks, so a latched jump/rail/freeze is cut but a brief transient that recovers is kept whole. This locator is independent of `is_surge_spec` (they share only the per-chunk statistics); its persistence applies wherever it runs, while a live-flagged boundary is taken as-is. Either way the prefix is then re-judged `CLEAN`/`SURGE`/`DEAD` and gets a bare, `_SURGE`, or no-file (`DEAD`) result — there is no `_JUMP` / `_RAIL` file; the jump/rail trigger is recorded only in the ledger's `event` column.

A.2 The three classification axes

Every acquisition is numbered in order and classified on three independent axes:

- **`event`** — the trigger / provenance: `JUMP`, `RAIL`, `FROZEN`, `DEAD`, `BAD_BASELINE`, or `NONE`. It is kept even when the saved prefix is clean, so a slip is never silently lost.
- **`outcome`** — the integrity of the (already-truncated) prefix: `CLEAN`, `SURGE`, `DEAD`, or `BAD_BASELINE`. `SURGE` is reserved for this axis (the post-hoc `is_surge_spec` verdict).
- **`accepted`** — whether the trace counts toward `n_trials`: true only when `outcome=CLEAN` *and* usable points $\geq$ `min_usable_frac` $\times$ `n_points`.

The saved file's point-count tag is the trace's *usable* length (e.g. `8p4Mpts`), and its only suffix is none, `_SURGE`, or `_BADBASE` — the `event` is *not* in the filename. A `DEAD` trace is logged but no file is written.

A.3 Event-by-event meaning

- **Cleared** — after a reset, the 1000-point verify read shows the locked baseline back near zero (|mean| < `baseline_v`, i.e. $< 0.1\,\mathrm{V}$): the latched integrator discharged and the flux lock recovered, so acquisition begins. `reset_and_verify` fires up to five resets and prints `CLEARED` on the first that passes. The action log records the clearing count (for example, `RESET,2 tries`); a failure records `RESET_FAIL,not cleared`.
- `outcome=CLEAN` — the saved prefix passed the post-hoc `is_surge_spec` check (no rail, opening mean in range, no $> 6\sigma$ baseline excursion, no flat/dead trace, no stuck/frozen segment). If the prefix is full-length (`event=NONE`) it is `accepted`, the clean count advances, and no reset fires — the lock is good. If it was truncated (`event=JUMP/RAIL/FROZEN`) but still clean, it is `accepted` only when its usable points reach `min_usable_frac` × `n_points`, and a reset clears the slipped lock before the next attempt either way.
- `event=JUMP` — a chunk's mean slipped from the established baseline by more than `jump_v`. Caught *live* mid-run (with `live_jump_check=True`) *or* located post-hoc on a flagless completion; either way the offending chunk is dropped and the clean prefix before it is kept — saved tagged by its usable length, **with no `_JUMP` suffix**; the trigger is recorded only as `event=JUMP` in the ledger, and the prefix's `outcome` is then `CLEAN` or `SURGE`. At $\pm 1\,\mathrm{V}$ a slip that reaches the clip reads as a $\sim 1\,\mathrm{V}$ jump, so it lands here rather than as a rail.
- `event=RAIL` — a chunk's $|V|_{\max}$ exceeded `rail_v` — a true rail (off-scale / lost lock). Caught live or located post-hoc, and handled exactly like a jump (drop the chunk, keep the clean prefix, `event=RAIL` in the ledger, no suffix file). Dormant at $\pm 1\,\mathrm{V}$ (the ADC clips below `rail_v`, so a clipped slip is a jump); active only if the card returns to $\pm 10\,\mathrm{V}$.
- `outcome=SURGE` — the saved prefix (truncated at a live flag, or located post-hoc) failed the post-hoc full-trace `is_surge_spec` check. That check flags *any* of: a rail ($|V|_{\max} >$ `rail_v`); an opening baseline already off-zero (first-chunk mean > `baseline_v`); a flat/dead trace (no chunk shows real noise); a stuck/frozen run (more than 2% of chunks have std below 10% of the live noise level); or a $> 6\sigma$ baseline excursion. Saved with `_SURGE`, reset, re-acquire.
- `outcome=DEAD` — the located prefix has *zero* usable points (a flat/dead trace whose robust noise level is below threshold). A flatline does not trip the live jump/rail check, so this is found by the post-hoc locator on *any* acquisition that finished without a live flag (live checking on or off). There is nothing to save, so **no trace file is written** — only a ledger row (`event=DEAD`, `outcome=DEAD`, `accepted=0`). The loop then resets and re-acquires like any other slip.
- `outcome=BAD_BASELINE` **/ reset is not holding** — a *live-check* outcome, detected in the first chunk(s), *before* the baseline is set: the trace started off-zero (|mean| > `baseline_v`) or railed, i.e. the reset did not actually hold although its short verify passed. Treated as systemic — the diagnostic is saved (`_BADBASE`) and logged, then the sweep stops immediately, no retry. Fix the reset and re-run (it resumes from disk). With `live_jump_check=False` there is no live baseline-window check, so an off-zero opening is instead caught post-hoc and saved as `_SURGE` rather than `_BADBASE`.
- **Done / Budget exhausted** — the interval reached `n_trials` *accepted* traces, or used all `max_attempts` real acquisitions with fewer (the lock kept slipping); either way it moves to the next interval (re-run to top up, or raise `max_attempts`).

A.4 How the run's counters relate

The header on each acquisition shows three quantities, each determined differently:

- **order index `i`** — the highest index already on disk for this trace's core (`interval_temp`, *ignoring* the month-day prefix *and* the point-count tag) plus one, then +1 on every acquisition. It is read from the filesystem, so it *continues* across re-runs, stops, and days; it is never reset or reused, and it is what the ledger's `attempt` column and the trailing filename number record (an accepted run after two slips is still `_3`).
- **`used` (out of `max_attempts`)** — real acquisitions consumed *this run* (any acquisition except a bad-baseline abort and the resets). A per-run counter that starts at zero each re-run; the loop stops at `max_attempts`.
- **`clean` (out of `n_trials`)** — the displayed progress counter; it counts *accepted* traces (a `CLEAN` prefix reaching `min_usable_frac`), seeded from the ledger's accepted count for the core (Section 5) and incremented on each accepted trace; the interval is done at `n_trials`.

The loop continues while `clean` $<$ `n_trials` *and* `used` $<$ `max_attempts`, so `index` $\geq$ `clean`: `index` advances on every acquisition, while `used` and `clean` advance only on the qualifying ones.

B Worked console and ledger example

A run prints something like the following — note the measurement start time and, for every attempt, that attempt's start and expected-done time. The classify reason printed in parentheses is the *prefix's* verdict, so a clean truncated prefix shows `(ok)` even though its `event` is `JUMP`:

```
temperature label for this run: 15mK
measurement START : 2026-05-31 14:00:00
  reset try 1: mean=+0.0021 V -> CLEARED

=== DAQ_Jun01_100us_15mK_10Mpts · index 1 · 0/4 used · 0/2 clean · 10000000 pts @ 10000 Hz (~1000 s) ===
    start 14:00:01 · expected done ~14:16:41  (~1000 s)
  event=NONE outcome=CLEAN (ok) · idx 1 · usable=10000000 pts (1000s) · accepted=True · 1001s · 1/2 clean

=== DAQ_Jun01_100us_15mK_10Mpts · index 2 · 1/4 used · 1/2 clean · 10000000 pts @ 10000 Hz (~1000 s) ===
    start 14:16:43 · expected done ~14:33:23  (~1000 s)
  event=JUMP outcome=CLEAN (ok) · idx 2 · usable=8430000 pts (843s) · accepted=False · 845s · 1/2 clean
  reset try 1: mean=+0.0019 V -> CLEARED

=== DAQ_Jun01_100us_15mK_10Mpts · index 3 · 2/4 used · 1/2 clean · 10000000 pts @ 10000 Hz (~1000 s) ===
    start 14:33:25 · expected done ~14:50:05  (~1000 s)
  event=JUMP outcome=CLEAN (ok) · idx 3 · usable=9500000 pts (950s) · accepted=True · 952s · 2/2 clean
  reset try 1: mean=+0.0020 V -> CLEARED       # exit reset: the final accepted prefix left the lock slipped
    DONE: 2/2 clean traces in 3 acquisition(s) this run.
```

Index 1 is a full clean trace (`accepted`); index 2 jumped live and was truncated to 8.43 M points — a `CLEAN` prefix, but below 0.90×10 M, so `accepted=False`; index 3 jumped at 9.5 M, which clears the threshold, so it is the second accepted trace. Because that final accepted prefix left the lock slipped, the *exit* reset fires before the interval returns.

Every line is also appended, one row per acquisition, to the run ledger `experiment_log.txt`. The timestamp is a full ISO date-time; an abbreviated sample of the columns (the full set is listed under *After the run*):

timestamp	event	outcome	usable_points	accepted	att	filename
2026-05-31T14:16:42	NONE	CLEAN	10000000	1	1	DAQ_Jun01_100us_15mK_10Mpts_1.txt
2026-05-31T14:33:24	JUMP	CLEAN	8430000	0	2	DAQ_Jun01_100us_15mK_8p4Mpts_2.txt
2026-05-31T14:50:06	JUMP	CLEAN	9500000	1	3	DAQ_Jun01_100us_15mK_9p5Mpts_3.txt

(`att` = `attempt`; the on-disk ledger also carries `usable_seconds`, `n_resets`, `mean_V`, `std_V`, and the start/end temperatures.)

Read `event` and `outcome` as separate axes: rows 2–3 carry `event=JUMP` (the jump trigger) while `outcome=CLEAN` (the truncated prefix is still good), and the filename uses the *usable* point tag (`8p4Mpts`, `9p5Mpts`) with *no* `_JUMP` suffix. `accepted` is the gate on `n_trials`: row 2 is below `min_usable_frac` so it does not count. A `DEAD` acquisition (flat/dead, found post-hoc on any flagless completion) would add a row with `event=DEAD`, `outcome=DEAD`, `usable_points=0`, `accepted=0`, and an *empty* filename — no trace file is written. A `BAD_BASELINE` row, by contrast, has a tiny `usable_points` and a mean near 1 V (the reset did not hold), is saved as `_BADBASE`, and stops the sweep.

References

[1] National Instruments, *PCIe-6320 Device Specifications*, NI document 374459 (16-bit multifunction DAQ, 250 kS/s).

[2] National Instruments, *NI-DAQmx Documentation* (analog-input tasks, finite sample-clock timing, terminal configuration); accessed via the `nidaqmx` Python API.

[3] STAR Cryoelectronics, *PFL-102 Programmable Feedback Loop and PCI-1000 Computer Interface — User's Manual* (SCC serial command protocol, feedback-loop register, DAC parameters).

[4] OpenSQUID project, `StarCryo` SCC reference implementation (GPLv3); source of the command framing, reset sequence, and `zeroStage1Output` centering routine reproduced here.

[5] STAR Cryoelectronics, *PCS102DA acquisition software and the PCS102 trace-file format* (metadata header + two-column time series).